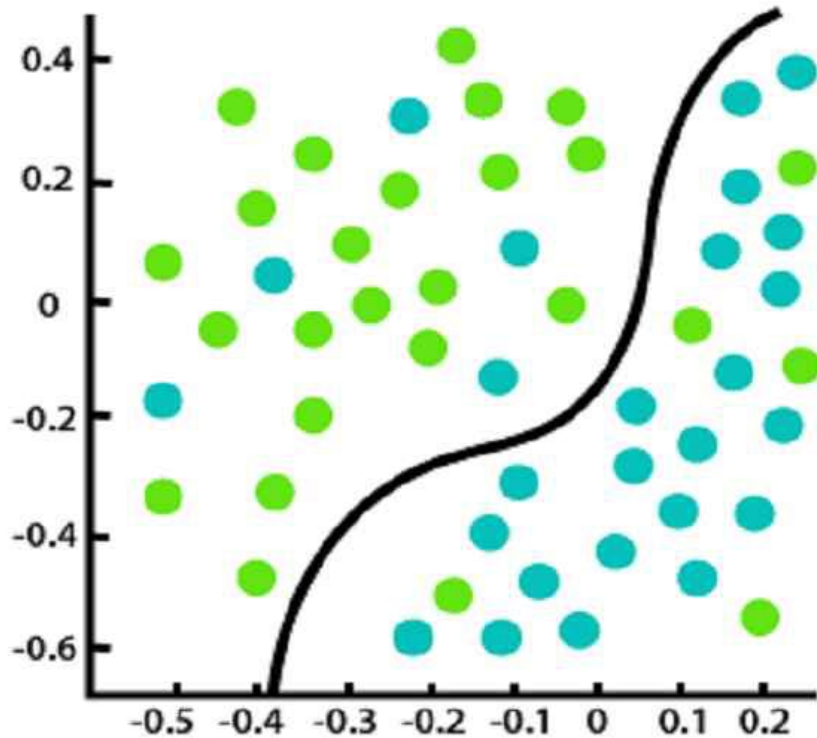


AMA 564 Deep Learning

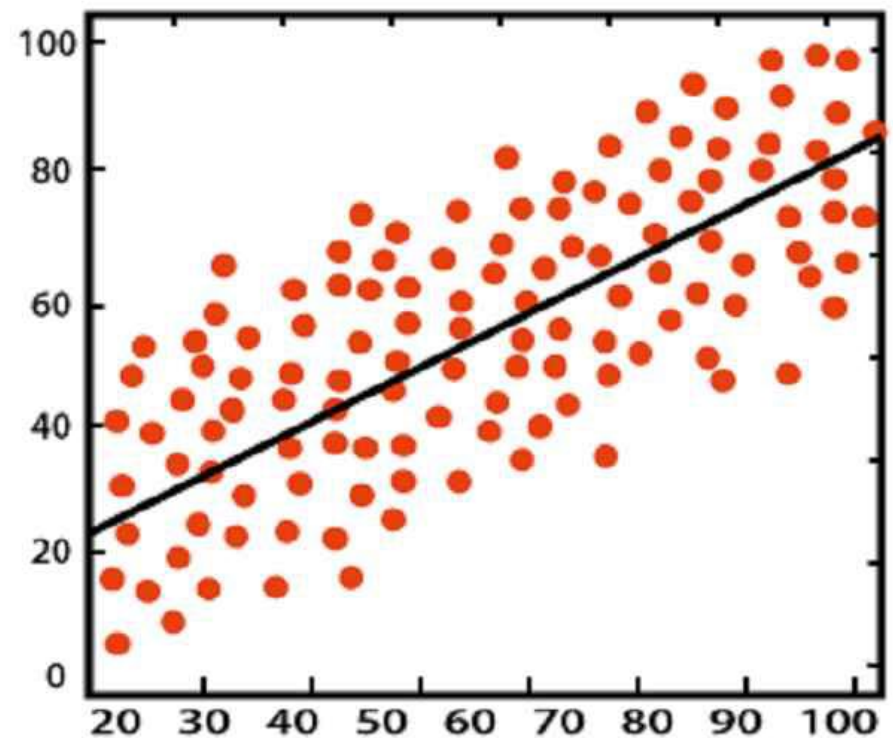
2026 Spring

Lecture 6

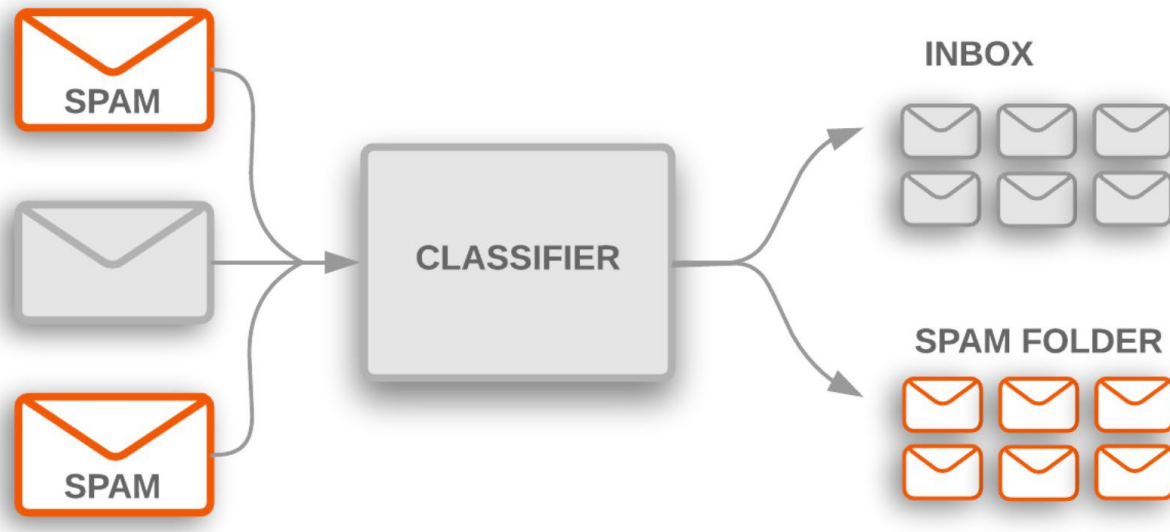
- 1. Classification Revisit**
- 2. Binary Classification**
- 3. Multi-class Classifications**



Classification

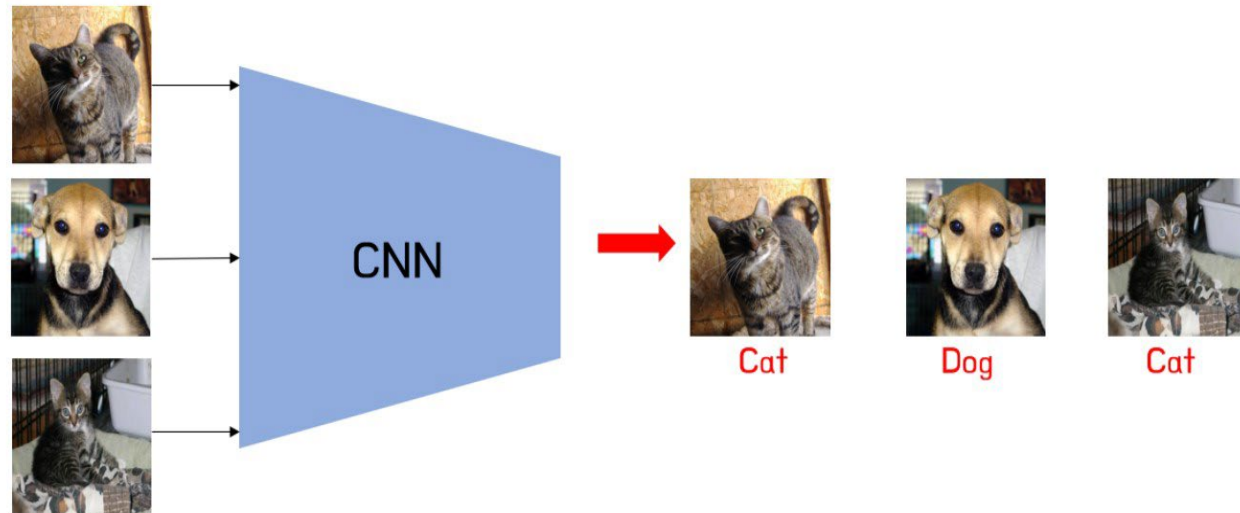


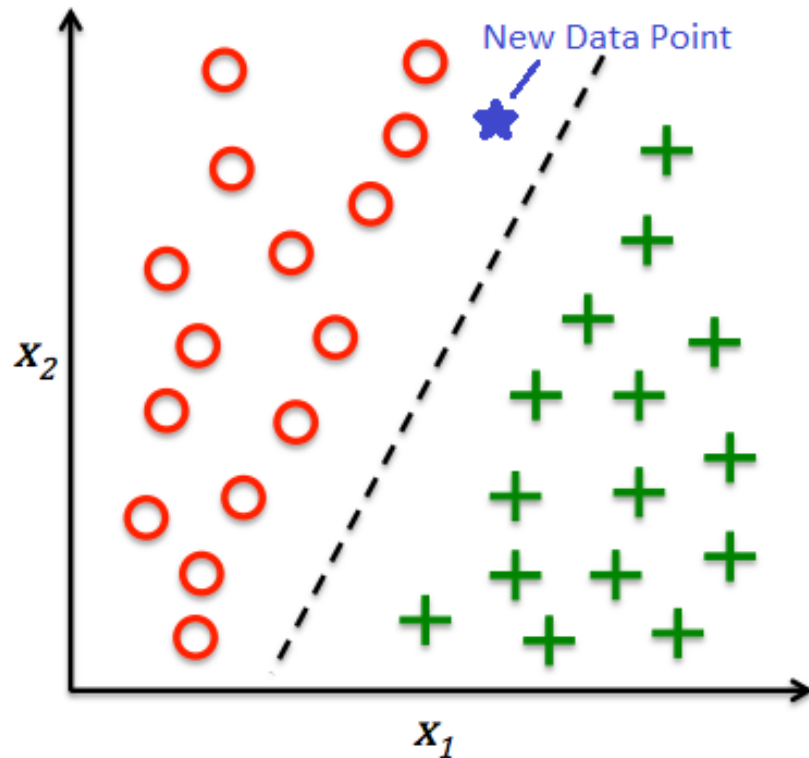
Regression



Spam Email Classification

Detection and Classification





- Data $(X_i, Y_i), i = 1, \dots, n$ i.i.d drawn from a distribution of (X, Y) where $Y_i \in \{1, 0\}$ (or $Y_i \in \{1, -1\}$).
- To find a function $h(\cdot)$ to maximize $\mathbb{P}(h(X_0) = Y_0)$ or equivalently to minimize $\mathbb{P}(h(X_0) \neq Y_0)$ over $\mathcal{H} = \{h: h(\cdot) \in \{1, 0\}\}$

There are many ways to do classification:

- Logistic Regression
- Support Vector Machine
- Deep Neural Networks

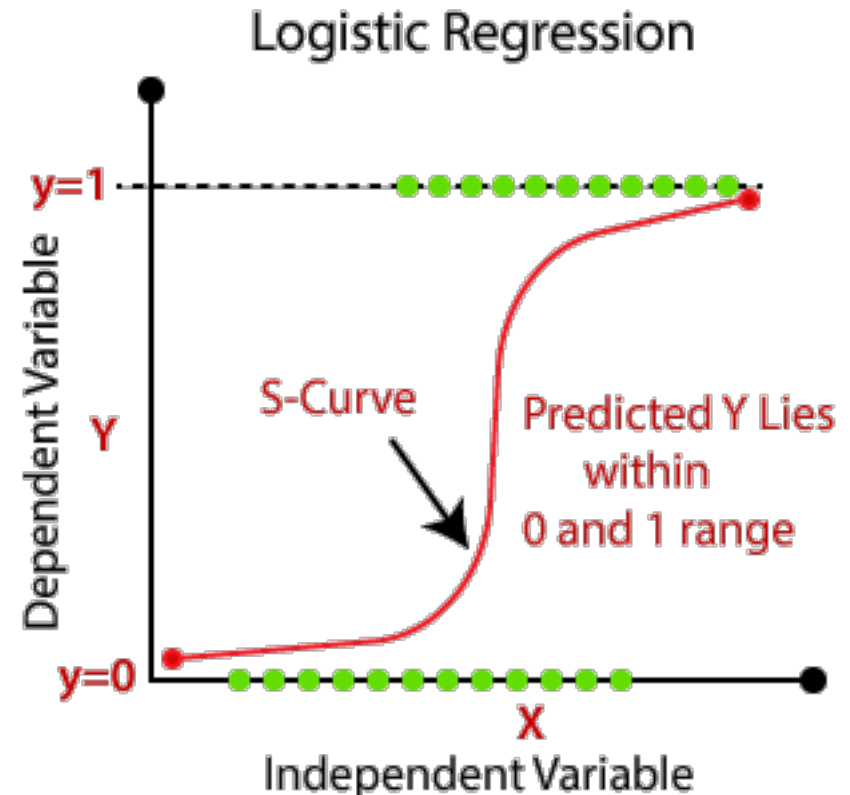
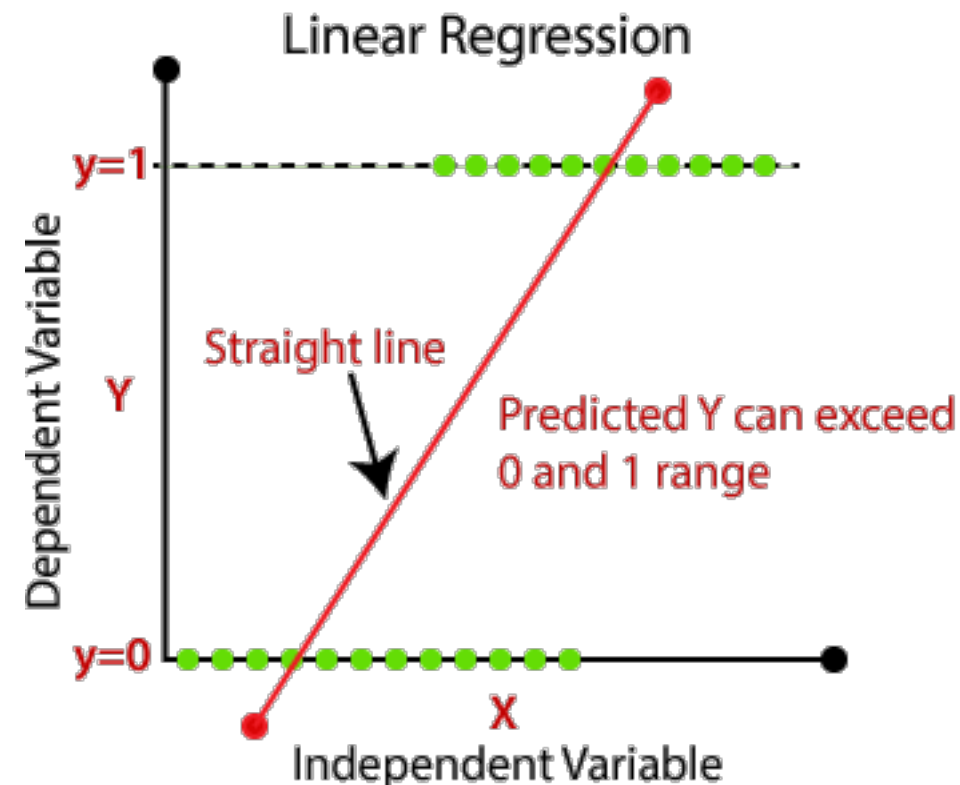
Brief review

on

Classification methods

Logistic Regression

Note: you can still use Least square regression.



Recall that the target for least square is **conditional mean**:

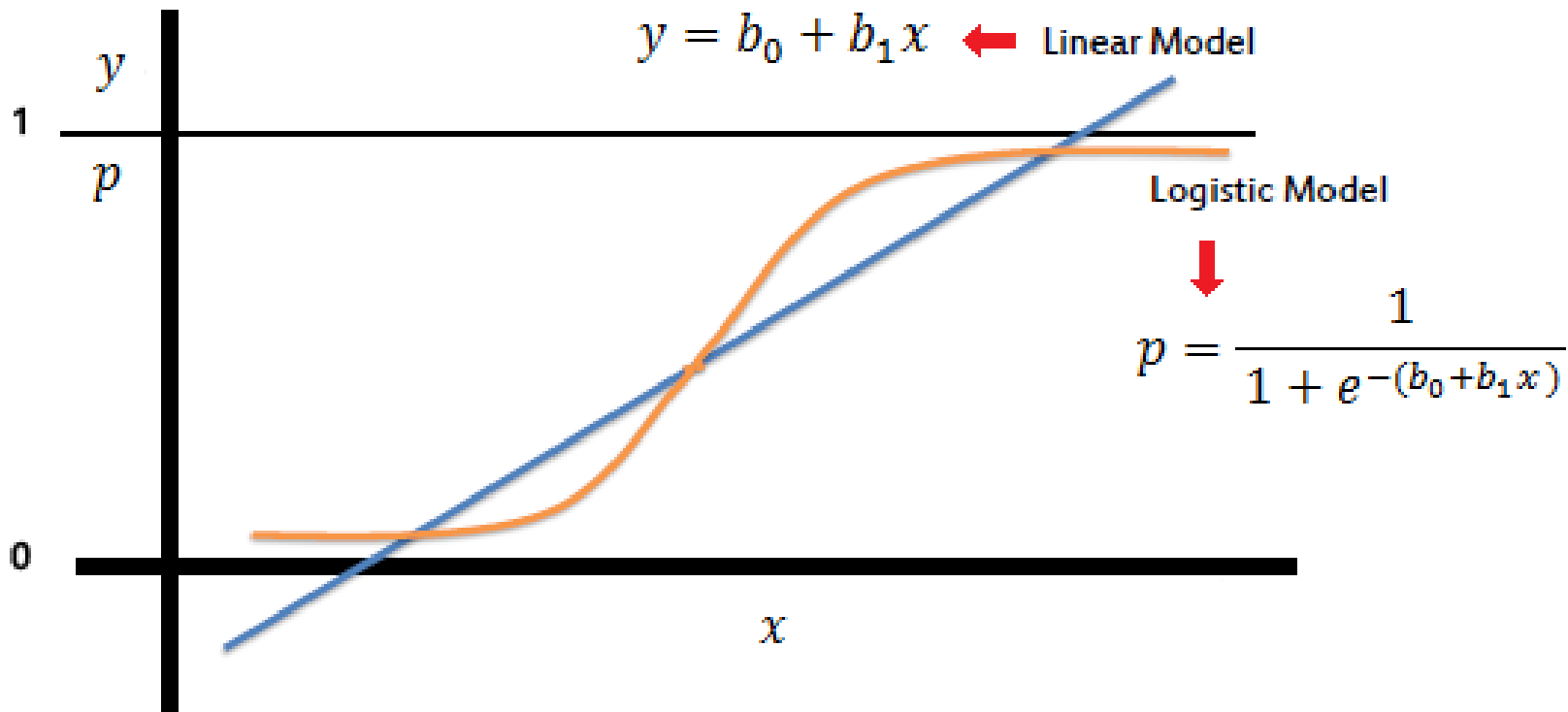
$$\mathbb{E}\{Y|X\} = 1 \times \mathbb{P}\{Y = 1|X\} + 0 \times \mathbb{P}\{Y = 0|X\} = \mathbb{P}\{Y = 1|X\}$$

It turns out to be the conditional probability of $Y = 1$ given X .

Logistic Regression

Logistic Regression

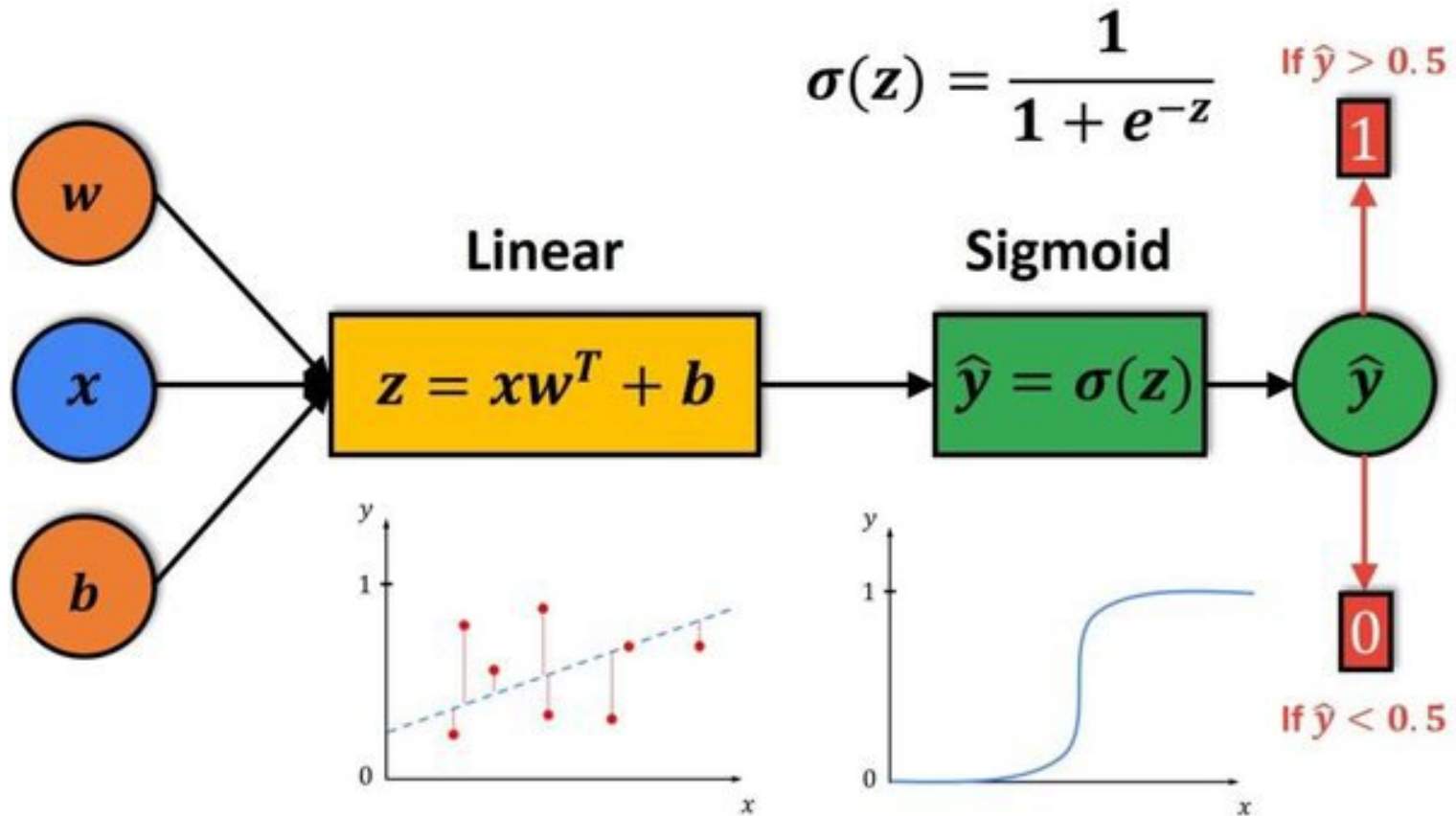
Logistic Regression models the conditional probability $p = \mathbb{P}\{Y = 1|X\}$



Logistic Regression targets at the estimation of b_0 and b_1 (univariate X)

Logistic Regression

Logistic Regression



Given a learned logistic model with parameters (w, b)

We can calculate $\hat{y} = \mathbb{P}\{Y = 1|X = x\}$ corresponding to a sample x

Predict its label $Y = 1$ if $\hat{y} > 0.5$, otherwise predict its label $Y = 0$.

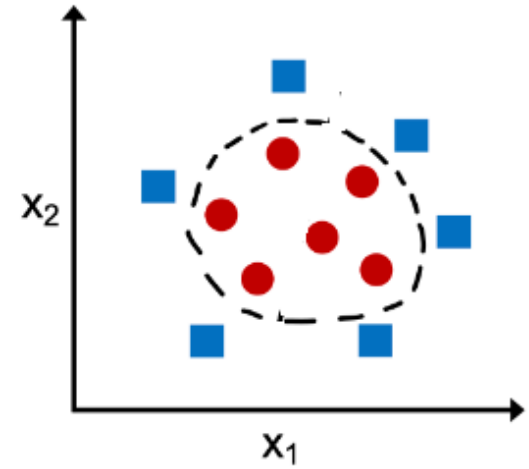
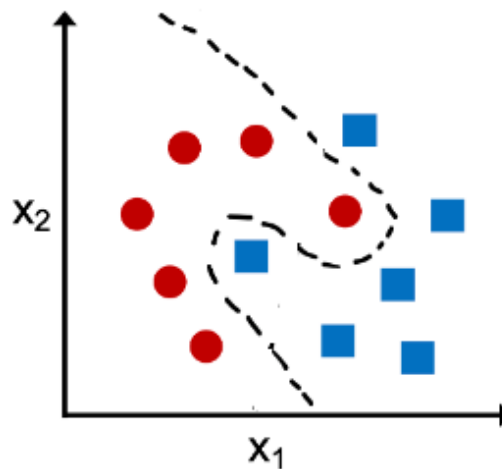
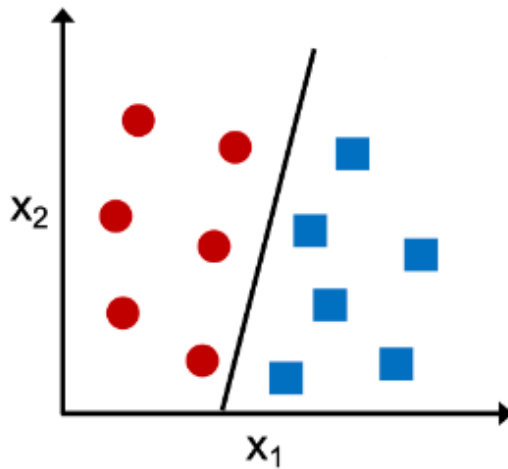
Logistic Regression

- Logistic regression models the conditional probability

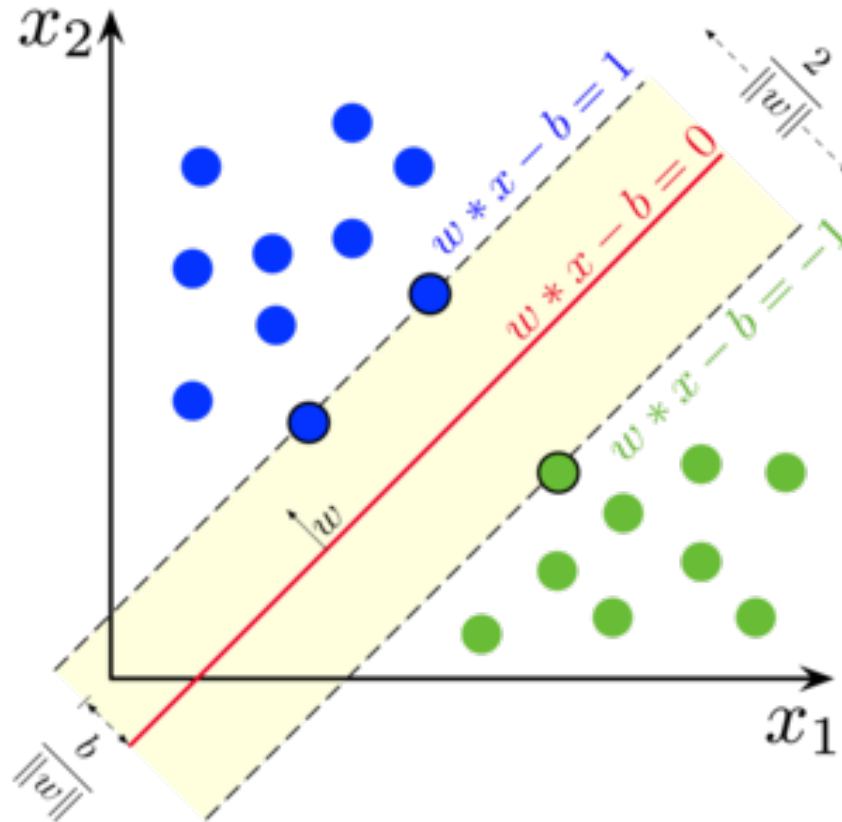
$$\mathbb{P}\{Y = 1|X\} = \frac{\exp(w'X + b)}{1 + \exp(w'X + b)}$$

- Classify a sample x into category 1 if $\mathbb{P}\{Y = 1|X = x\} > 0.5$
- Classify a sample x into category 0 if $\mathbb{P}\{Y = 1|X = x\} < 0.5$
- The decision boundary is linear $\{x: w'x + b = 0\}$
- Not suitable for data sets with non-linear boundary

In many cases, the data is not **linearly separable**!

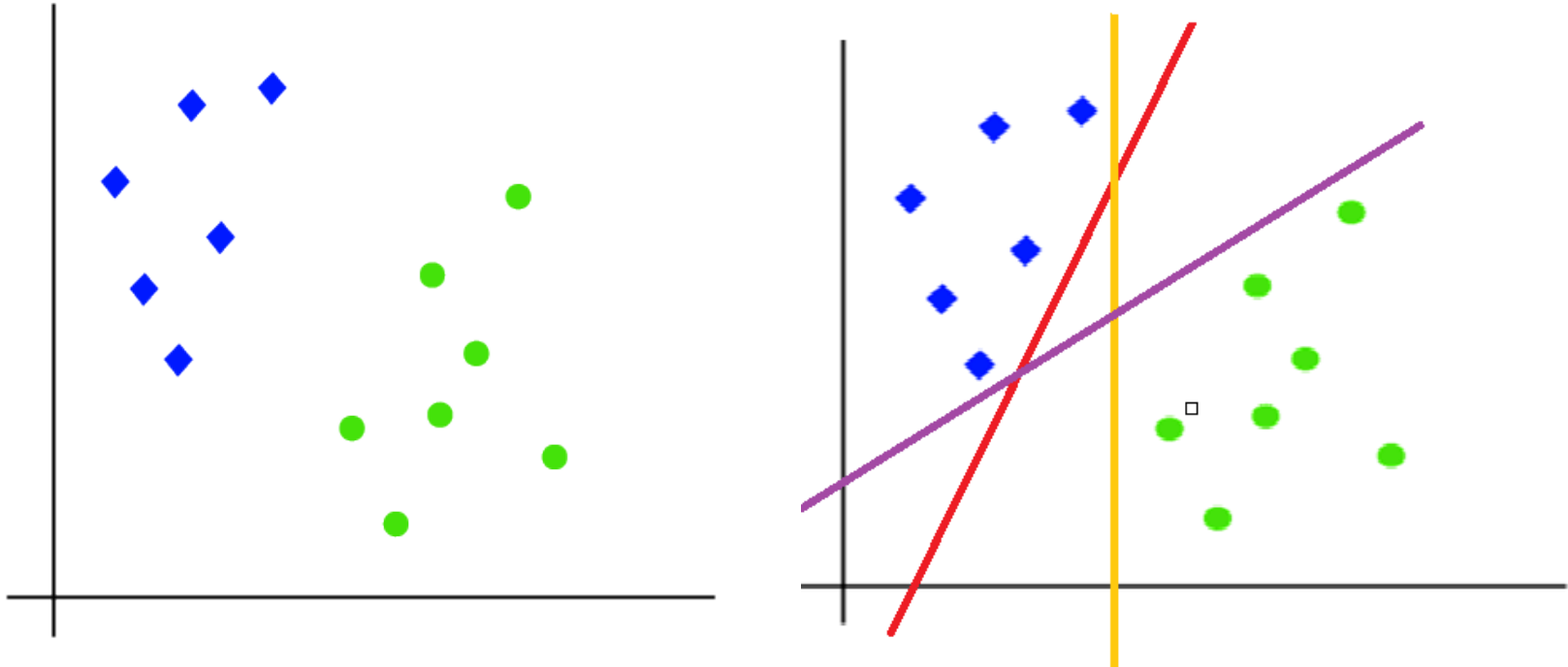


Source: <https://vitalflux.com/classification-problems-real-world-examples/>



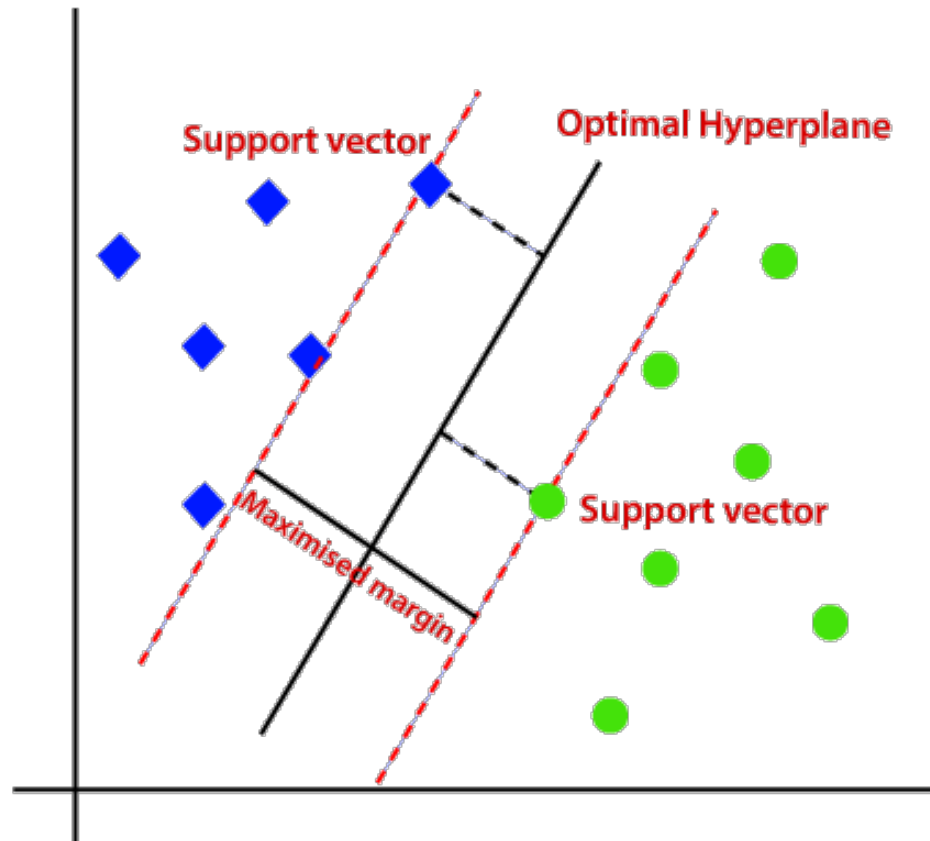
Source: https://en.wikipedia.org/wiki/Support_vector_machine

Support vector machine aims to find a hyperplane with the largest **margin** to separate different class samples



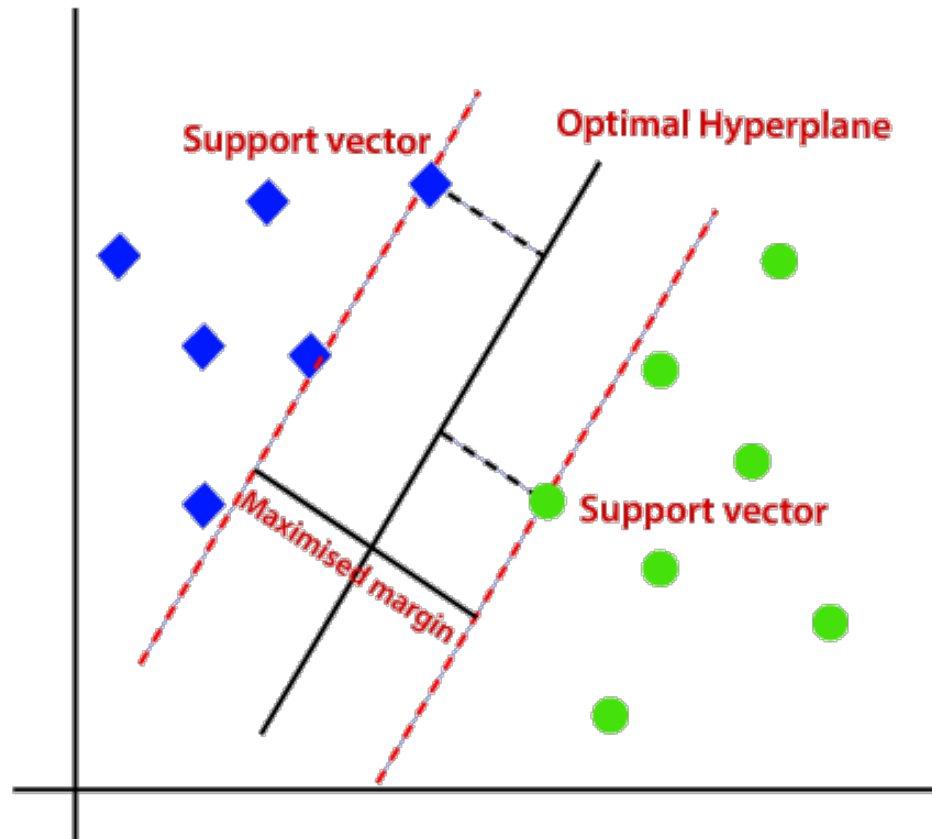
Source: <https://www.analyticsvidhya.com/blog/2021/10/support-vector-machinessvm-a-complete-guide-for-beginners/>

There could be many hyperplanes that can separate different class samples.



Source: <https://www.analyticsvidhya.com/blog/2021/10/support-vector-machinessvm-a-complete-guide-for-beginners/>

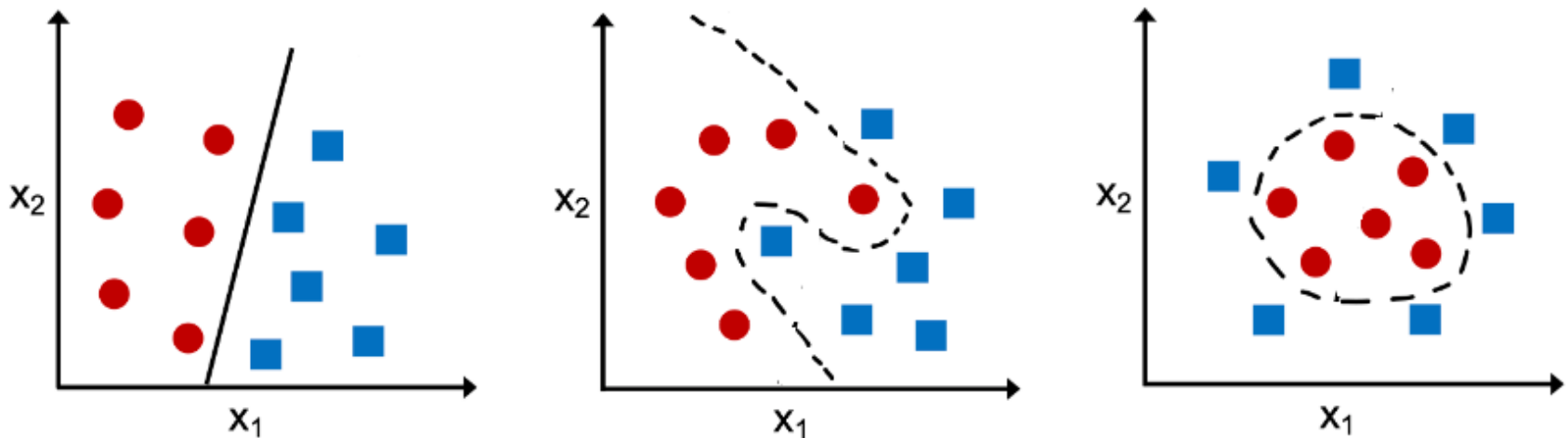
The **best hyperplane** is that plane that **has the maximum distance from both the classes**, and this is the main aim of SVM.



The **best hyperplane** only depends on a subset of the sample. These sample are called **support vectors**.

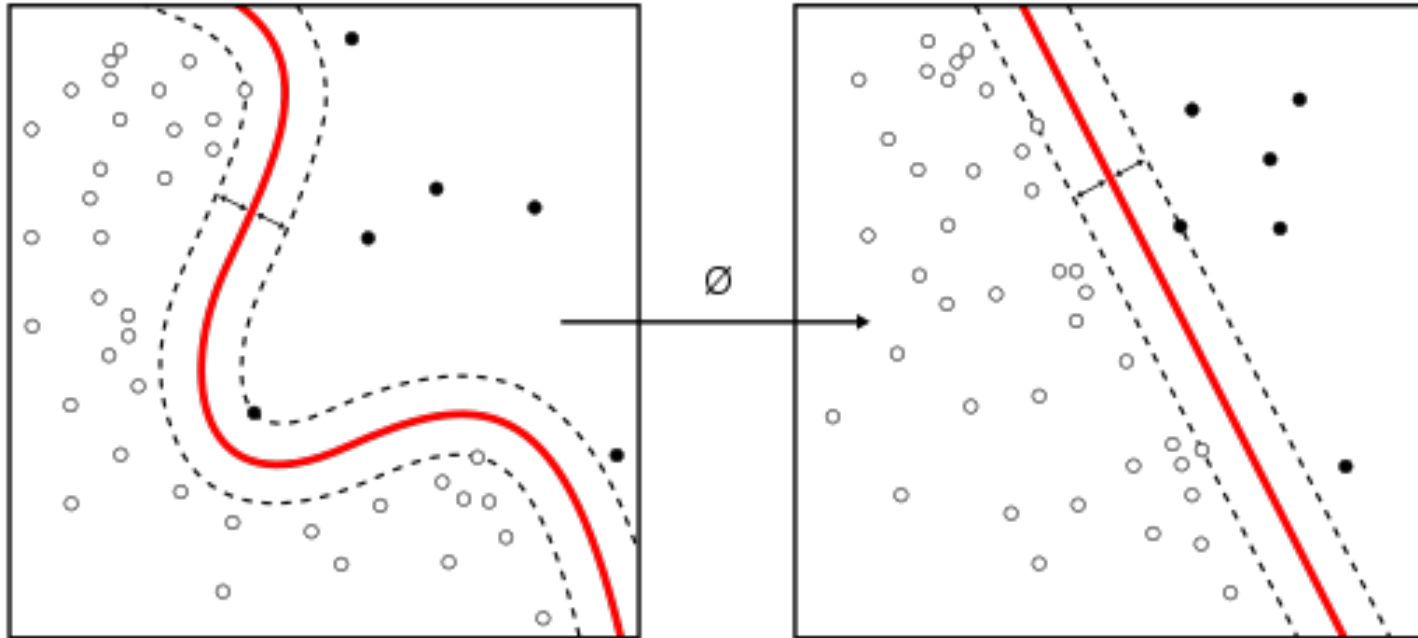
[Recommend Reference Materials](#)

Support Vector Machine is also a **linear classifier**.



Source: <https://vitalflux.com/classification-problems-real-world-examples/>

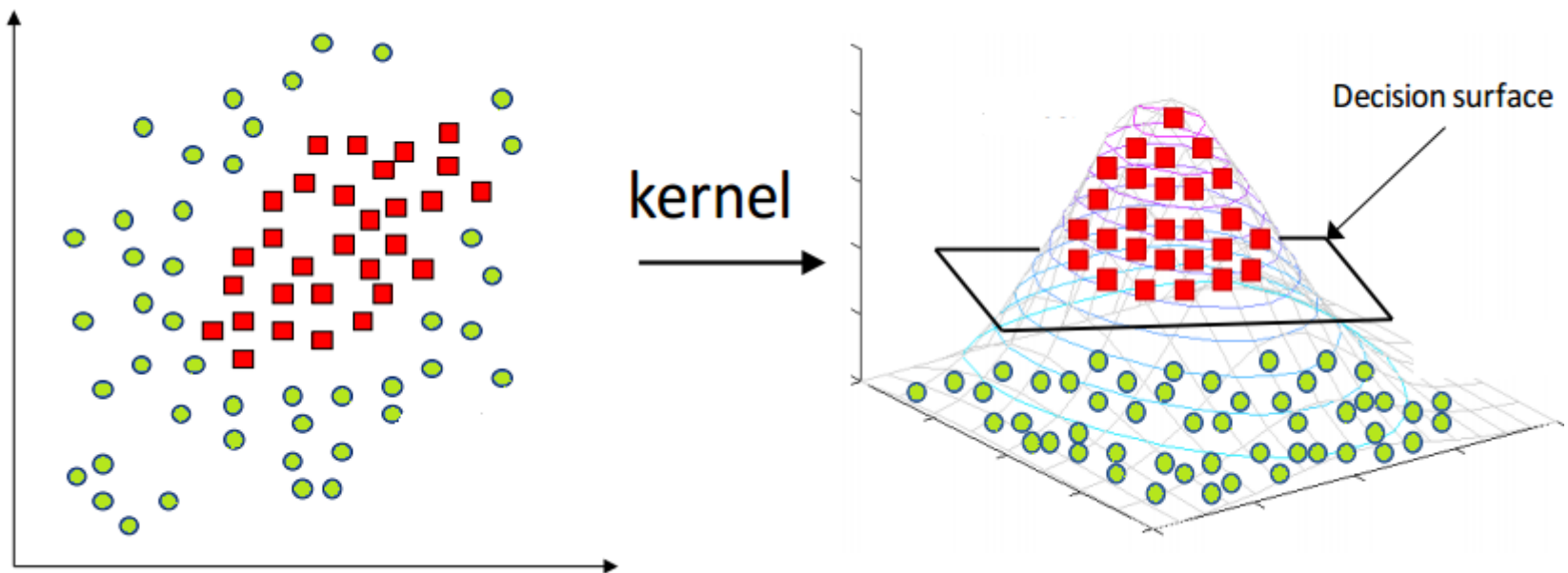
Support Vector Machine with Kernel Trick



Using **nonlinear** kernels allows the algorithm to fit the maximum-margin hyperplane in **a transformed feature space**.

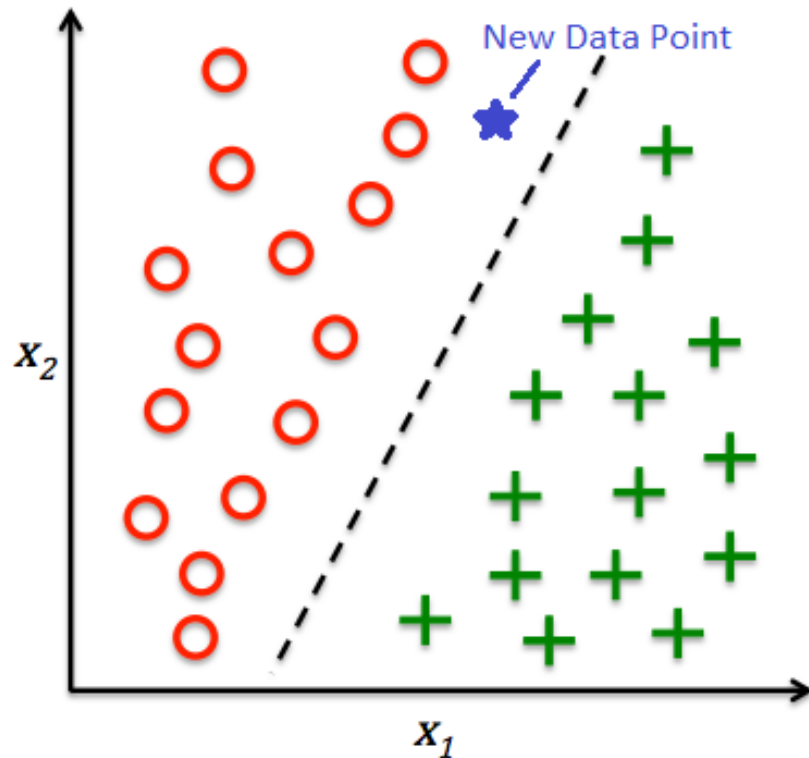
Although the classifier is a hyperplane in the transformed feature space, it may be nonlinear in the original input space.

Support Vector Machine with Kernel Trick



The **transformed feature space** can be a higher-dimensional space.

How to use Deep Neural Networks to do Binary classification?



- Data $(X_i, Y_i), i = 1, \dots, n$ i.i.d drawn from a distribution of (X, Y) where $Y_i \in \{1, -1\}$.
- To find a function $h(\cdot)$ to maximize

$$\mathbb{P}(h(X_0) = Y_0)$$
or equivalently to minimize

$$\mathbb{P}(h(X_0) \neq Y_0)$$
over

$$\mathcal{H} = \{h: h(\cdot) \in \{1, -1\}\}$$

- Empirical risk minimization:

to find a function $h(\cdot)$ to minimize

$$\frac{1}{n} \sum_{i=1}^n I(h(X_i) \neq Y_i)$$

over

$$\mathcal{H} = \{h: h(\cdot) \in \{1, -1\}\}$$

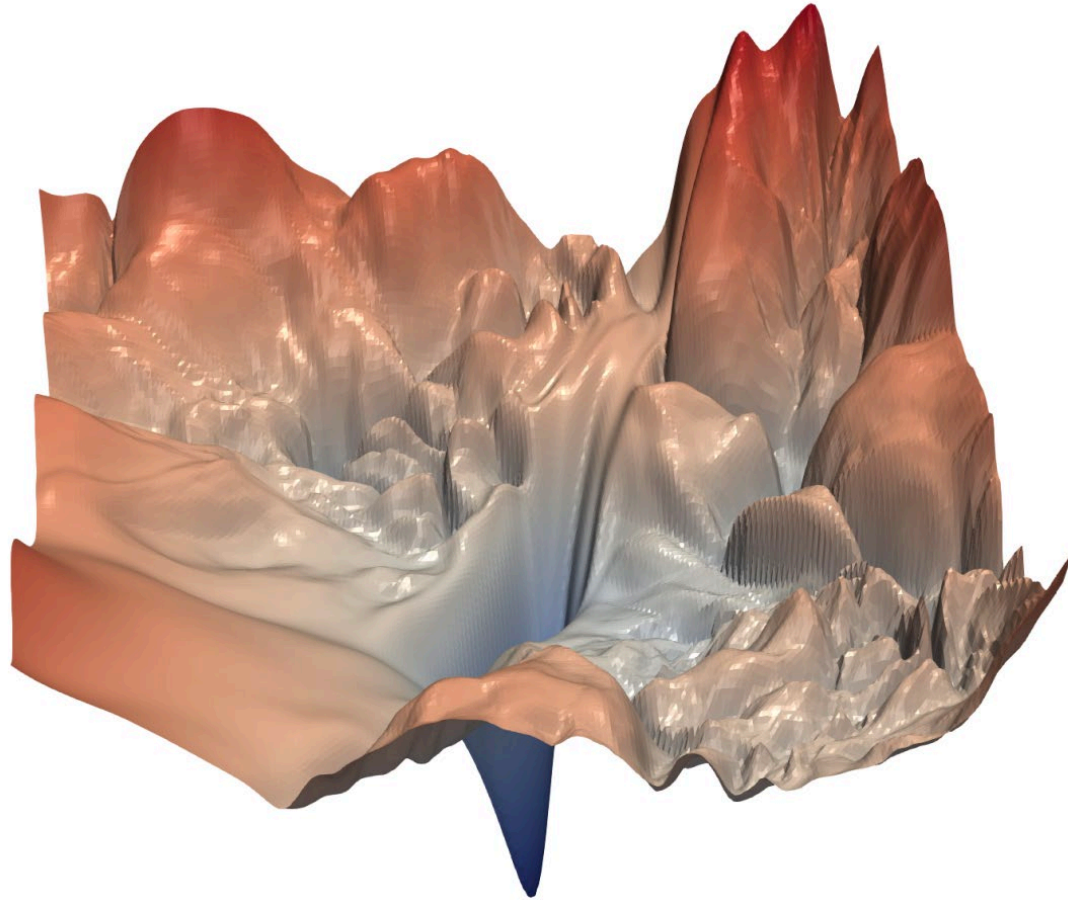
Empirical risk minimization: to find a function $h(\cdot)$ to minimize

$$\frac{1}{n} \sum_{i=1}^n I(h(X_i) \neq Y_i)$$

over

$$\mathcal{H} = \{h: h(\cdot) \in \{1, -1\}\}.$$

- **0-1 Loss function** $I(\cdot)$: non-continuous, non-smooth
- **Classifier** $h(\cdot)$: not regular smooth function
- **The optimization problem is extremely hard !**



Source: <https://arxiv.org/pdf/1712.09913.pdf>

The loss surface under 0-1 loss can be extremely complicated !

The optimization problem is extremely hard !

Empirical risk minimization: to find a function $f(\cdot)$ to minimize

$$\frac{1}{n} \sum_{i=1}^n L(f(X_i, \theta), Y_i)$$

over

$\mathcal{F} = \{f: f(x; \theta) \text{ is a neural network}$
 $\text{parameterized by } \theta \in \mathbb{R}^s \text{ outputs real values}\}.$

We expect

- **Surrogate Loss function** $L(\cdot, \cdot)$: continuous, smooth
- **Neural network** $f(\cdot; \theta)$: output continuous value
- **The estimation easy to implement and explain**

Empirical risk minimization: to find a function $f(\cdot)$ to minimize

$$\frac{1}{n} \sum_{i=1}^n L(f(X_i, \theta), Y_i)$$

over

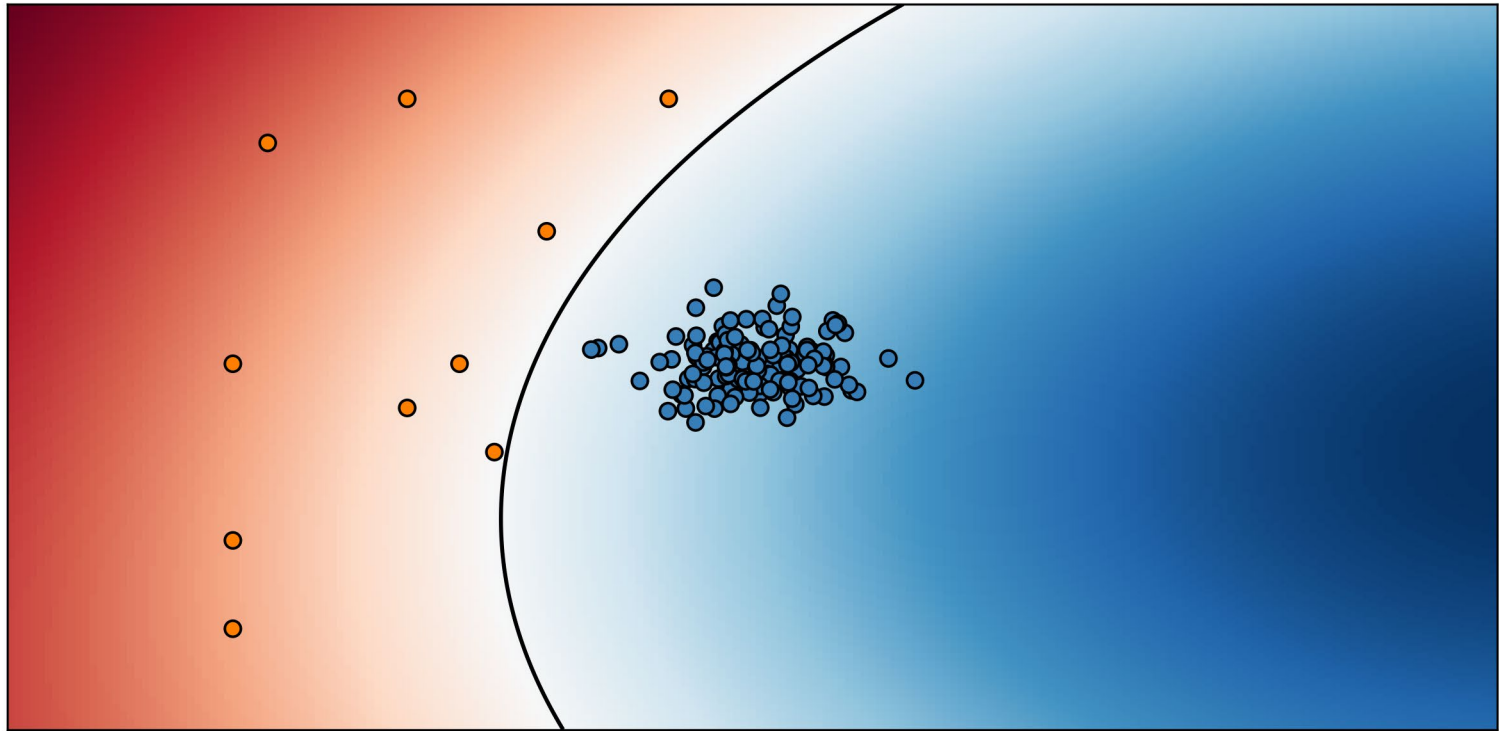
$\mathcal{F} = \{f: f(x; \theta) \text{ is a neural network}$
 $\text{parameterized by } \theta \in \mathbb{R}^s \text{ outputs real values}\}.$

Idea: use threshold value to create a classifier

$$h(X_i, \theta) = \text{sgn}\{f(X_i, \theta)\}$$

If $f(x_0; \theta) > 0$, then predict $\hat{y}_0 = +1$;

If $f(x_0; \theta) < 0$, then predict $\hat{y}_0 = -1$.



Source: <https://www.flair-tech.com/en/why-anomaly-detection-is-not-binary-classification/>

Idea: use threshold value to create a classifier

$$h(X_i, \theta) = \text{sgn}\{f(X_i, \theta)\}$$

If $f(x_0; \theta) > 0$, then predict $\hat{y}_0 = +1$;

If $f(x_0; \theta) < 0$, then predict $\hat{y}_0 = -1$.

Empirical risk minimization: to find a function $f(\cdot)$ to minimize

$$\frac{1}{n} \sum_{i=1}^n \phi(f(X_i, \theta) \times Y_i)$$

where $\phi(\cdot)$ in general is continuous and decreasing.

$\phi(\cdot)$ is the surrogate loss function.

Idea:

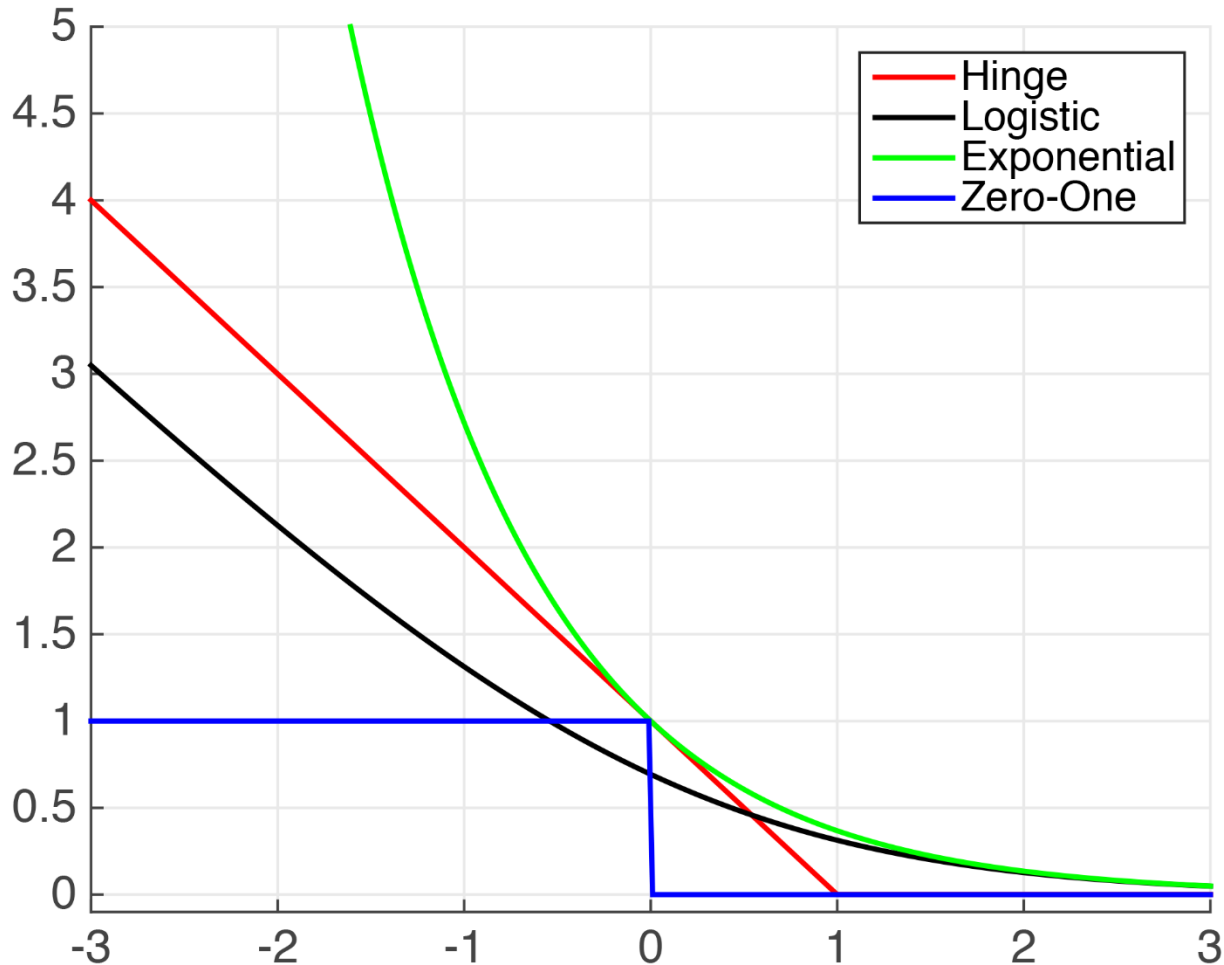
If label $Y_i = +1$, then larger positive $f(X_i; \theta) > 0$ decreases the loss,

and it predicts $\hat{Y}_i = +1$.

If label $Y_i = -1$, then smaller negative $f(X_i; \theta) < 0$ decreases the loss.

and it predicts $\hat{Y}_i = -1$.

Feasible loss functions



0-1 loss:

$$\phi(y \cdot f(x, \theta)) = I(y \cdot f(x, \theta) < 0)$$

Exponential loss (AdaBoost): $\phi(y \cdot f(x, \theta)) = \exp(-y \cdot f(x, \theta))$

Logistic loss :

$$\phi(y \cdot f(x, \theta)) = \log\{1 + \exp[-y \cdot f(x, \theta)]\}$$

Hinge loss (SVM):

$$\phi(y \cdot f(x, \theta)) = \max\{1 - y \cdot f(x, \theta), 0\}$$

- Given a surrogate loss ϕ , let $\hat{f}_n = \operatorname{argmin}_n \frac{1}{n} \sum_{i=1}^n \phi(f(X_i, \theta) \times Y_i)$
- Let $\hat{h}_n = \operatorname{sgn}(\hat{f}_n)$ be a classifier based on $\hat{f}_n$
- Then $\hat{h}_n$ is a good classifier in the sense that when sample size n is very large,

$$\hat{h}_n \approx \operatorname{argmin}_n \frac{1}{n} \sum_{i=1}^n I(h(X_i) \neq Y_i)$$

Proper surrogate loss function will lead to a consistent classifier.

Suggested Reference:

Bartlett, P. L., Jordan, M. I., & McAuliffe, J. D. (2006). **Convexity, classification, and risk bounds**. *Journal of the American Statistical Association*, 101(473), 138-156.

How to use Deep Neural Networks to do Multi-class classification?

- For binary classification, label $Y = +1$ or $Y = -1$ stands for different classes.
- What if there are **more than 2** classes (categories) of the data?

$Y = +1$, $Y = 0$ and $Y = -1$?

$Y = 0, 1, 2, 3, 4, \dots$?

The Order matters !

Preprocess the data

Binary classification example:

X_i is an image

$$Y_i \in \{1, -1\}$$

Cat: $Y_i = -1$

Dog: $Y_i = +1$



X_i is an image

$$Y_i \in \{0, 1\}^2 \subset \mathbb{R}^2$$

Cat: $Y_i = (0, 1)$

Dog: $Y_i = (1, 0)$

Preprocess the data

Multi-class classification example:

X_i is an image

$$Y_i \in \{0, 1\}^K \subset \mathbb{R}^K$$

Dog: $Y_i = (1, 0, 0, \dots, 0)$

Cat: $Y_i = (0, 1, 0, \dots, 0)$

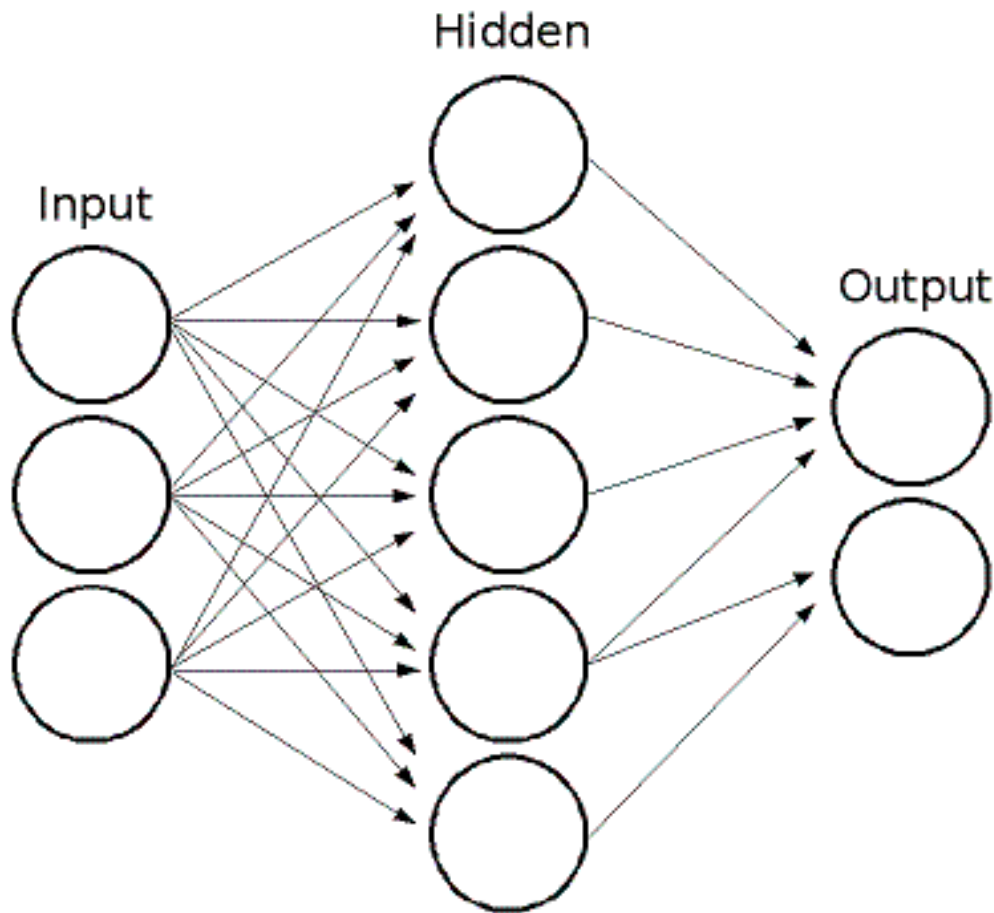
Frog: $Y_i = (0, 0, 1, \dots, 0)$

.....

Tiger: $Y_i = (0, 0, \dots, 0, 1)$

Adjust the output of neural network

Binary classification example:



*X_i is an image,
 $Y_i \in \{0, 1\}^2 \subset \mathbb{R}^2$*

Cat: $Y_i = (0, 1)$

Dog: $Y_i = (1, 0)$

We let $h: \mathbb{R}^d \rightarrow \mathbb{R}^2$, i.e.
 $h(x, \theta) = (z_1, z_2)$.

Adjust the output of neural network

Binary classification example:

X_i is an image, $Y_i \in \{0, 1\}^2 \subset \mathbb{R}^2$

Cat: $Y_i = (0, 1)$ Dog: $Y_i = (1, 0)$

- Let $h: \mathbb{R}^d \rightarrow \mathbb{R}^2$, i.e. $h(x, \theta) = (z_1, z_2)$.
- We hope $h(X_i, \theta) = (z_1, z_2) = (1, 0)$ if $Y_i = (1, 0)$,
and $h(X_i, \theta) = (z_1, z_2) = (0, 1)$ if $Y_i = (0, 1)$.
- We hope use $z_1, z_2 \in [0, 1]$ to model class probabilities.

Adjust the output of neural network

Binary classification example:

- Let $\mathbf{h}: \mathbb{R}^d \rightarrow \mathbb{R}^2$, i.e. $\mathbf{h}(x, \boldsymbol{\theta}) = (z_1, z_2)$. Note $z_1, z_2 \in \mathbb{R}$.
- Use an additional **SoftMax** layer to $\mathbf{h}$:

$$\text{SoftMax}(\mathbf{h}(x, \boldsymbol{\theta}))$$

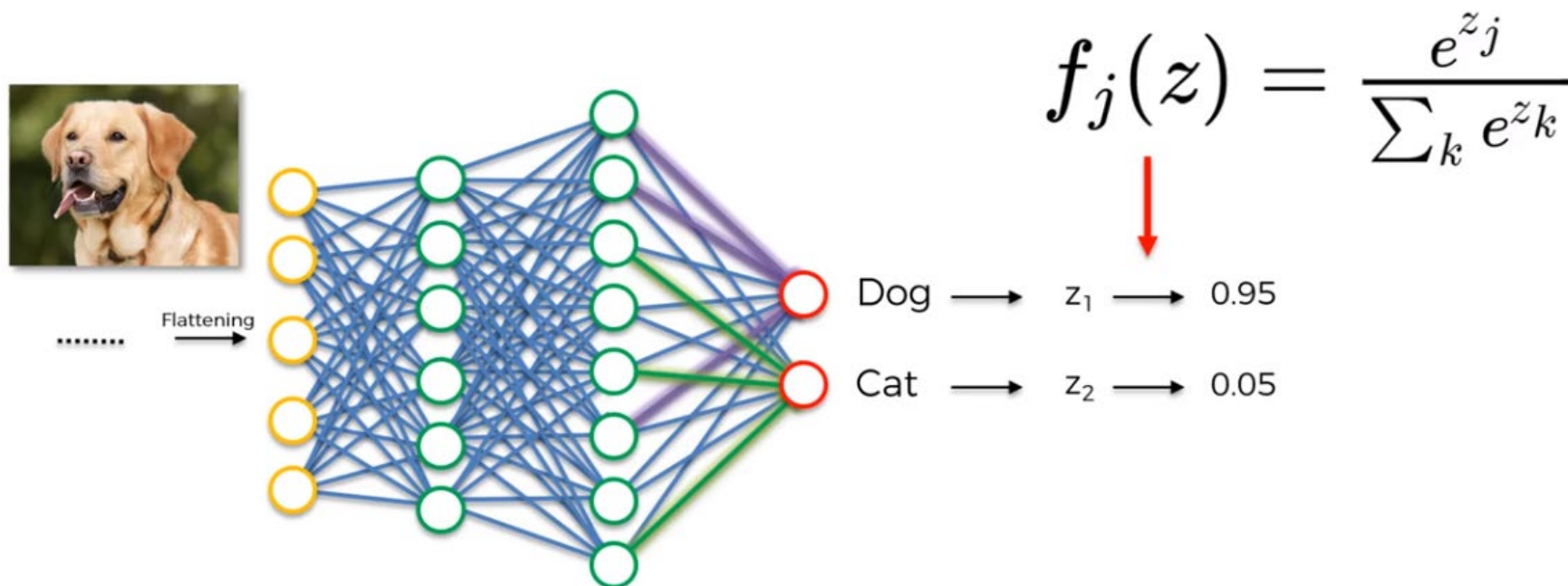
$$= \text{SoftMax}((z_1, z_2)) = (\widehat{y}_1, \widehat{y}_2)$$

$$= \begin{pmatrix} \frac{\exp(z_1)}{\sum_{i=1}^2 \exp(z_i)} \\ \frac{\exp(z_2)}{\sum_{i=1}^2 \exp(z_i)} \end{pmatrix}$$

- $\widehat{y}_1, \widehat{y}_2 \in [0, 1]$ can be the class probabilities.

Adjust the output of neural network

Binary classification example:



Source: <https://www.andreaperlato.com/aipost/cnn-and-softmax/>

$SoftMax(h(x, \theta)) = (\hat{y}_1, \hat{y}_2)$. Note $\hat{y}_1, \hat{y}_2 \in [0, 1]$ and $\hat{y}_1 + \hat{y}_2 = 1$.

Model $\mathbb{P}(Y_i = (1, 0) \mid X_i = x)$ by $\hat{y}_1$. Model $\mathbb{P}(Y_i = (0, 1) \mid X_i = x)$ by $\hat{y}_2$.

What is the Objective?

Binary classification example:

Model

$\text{SoftMax}(h(x, \theta)) = (\hat{y}_1, \hat{y}_2)$. Note $\hat{y}_1, \hat{y}_2 \in [0, 1]$ and $\hat{y}_1 + \hat{y}_2 = 1$.

$\text{SoftMax}(h(x, \theta))_1 = \hat{y}_1$ is the predicted value for $\mathbb{P}(Y_i = (1, 0) \mid X_i = x)$.

$\text{SoftMax}(h(x, \theta))_2 = \hat{y}_2$ is the predicted value for $\mathbb{P}(Y_i = (0, 1) \mid X_i = x)$.

Data

$$Y_i = (Y_{i1}, Y_{i2})$$

$Y_{i1} = 1$ implies $Y_i = (1, 0)$ and the picture is a dog.

$Y_{i2} = 1$ implies $Y_i = (0, 1)$ and the picture is a cat.

What is the Objective?

Binary classification example:

The Likelihood function

$$[\mathbb{P}(Y_i = (1, 0) \mid X_i = x)]^{I(Y_i=(1,0))} \times [\mathbb{P}(Y_i = (0, 1) \mid X_i = x)]^{I(Y_i=(0,1))}$$

$$\left[\text{SoftMax}(h(x, \theta))_1 \right]^{I(Y_i=(1,0))} \times \left[\text{SoftMax}(h(x, \theta))_2 \right]^{I(Y_i=(0,1))}$$

$$= [\hat{y}_1]^{I(Y_{i1}=1)} \times [\hat{y}_2]^{I(Y_{i2}=1)}$$

$$= [\hat{y}_1]^{Y_{i1}} \times [\hat{y}_2]^{Y_{i2}}$$

The log Likelihood function

$$Y_{i1} \times \log [\hat{y}_1] + Y_{i2} \times \log [\hat{y}_2]$$

What is the Objective?

Binary classification example:

Maximize the log Likelihood function

$$\max Y_{i1} \times \log [\hat{y}_1] + Y_{i2} \times \log [\hat{y}_2]$$

Minimize the negative log Likelihood function

$$\min -Y_{i1} \times \log [\hat{y}_1] - Y_{i2} \times \log [\hat{y}_2]$$

$$\min -\sum_{j=1}^m Y_{ij} \times \log \hat{y}_j$$

Loss function for classification

Cross Entropy loss function :

$$\text{Loss} = - \sum_{i=1}^{\text{output size}} y_i \cdot \log \hat{y}_i$$

For binary classification, **output size=2**

- Label **$\mathbf{y} = (y_1, y_2)$** , $y_1, y_2 \in \{0, 1\}$
- Prediction **$\text{SoftMax}(h(x, \theta)) = \hat{\mathbf{y}} = (\hat{y}_1, \hat{y}_2)$** , $\hat{y}_1, \hat{y}_2 \in [0, 1]$

Empirical risk minimization: to find a function $h(\cdot)$ to minimize

$$\frac{1}{n} \sum_{i=1}^n \text{CELoss}(h(X_i, \theta), Y_i)$$

over

$\mathcal{H} = \{h: h(x; \theta) \text{ is a neural network}$
 $\text{parameterized by } \theta \in \mathbb{R}^s\},$

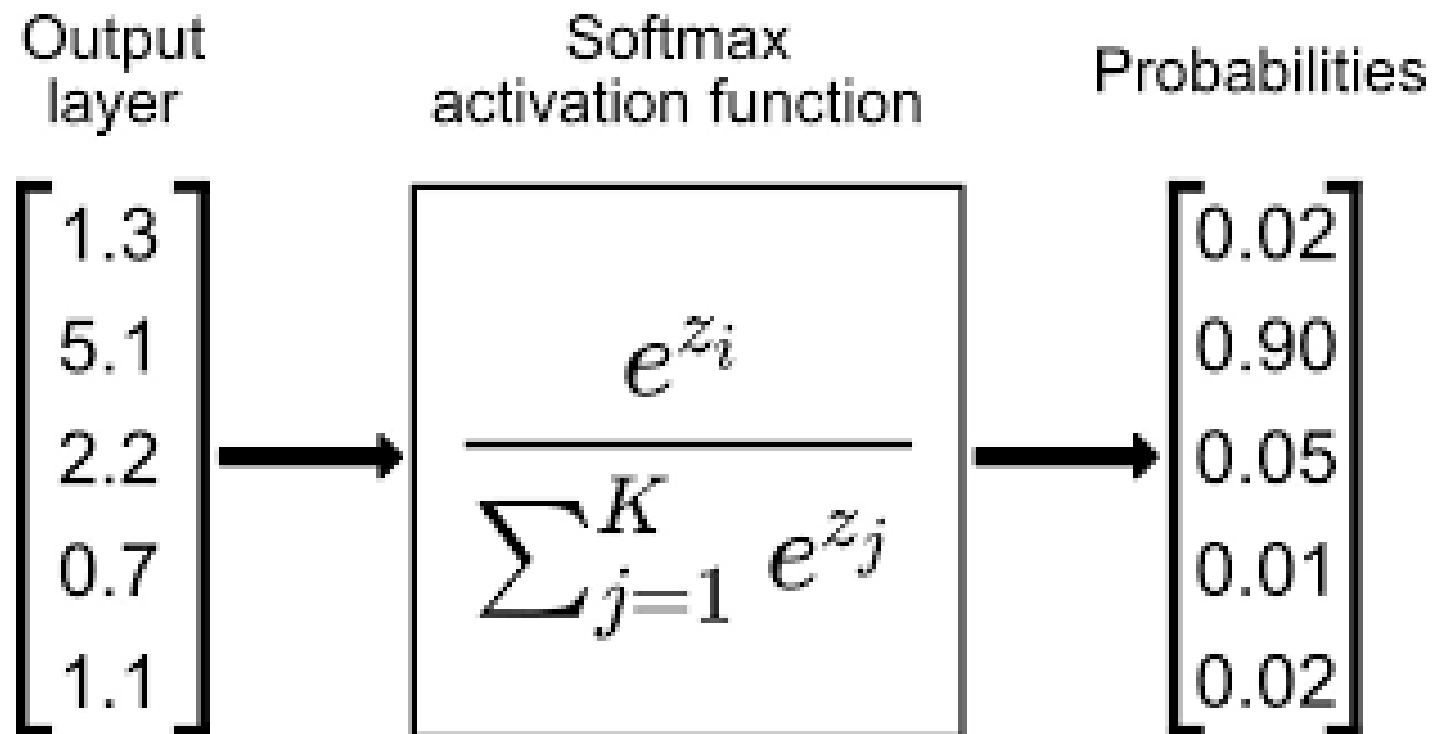
where

$$\text{CELoss}(h(X_i, \theta), Y_i) = \frac{1}{n} \sum_{i=1}^n \left[- \sum_{j=1}^m Y_{ij} \cdot \log \left\{ \frac{\exp(h(X_i, \theta)_j)}{\sum_{j=1}^m \exp(h(X_i, \theta)_j)} \right\} \right]$$

m is the output size of h or the number of classes (categories).

Apply to multi-class problem

SoftMax function:



Apply to multi-class problem

Cross Entropy loss function :

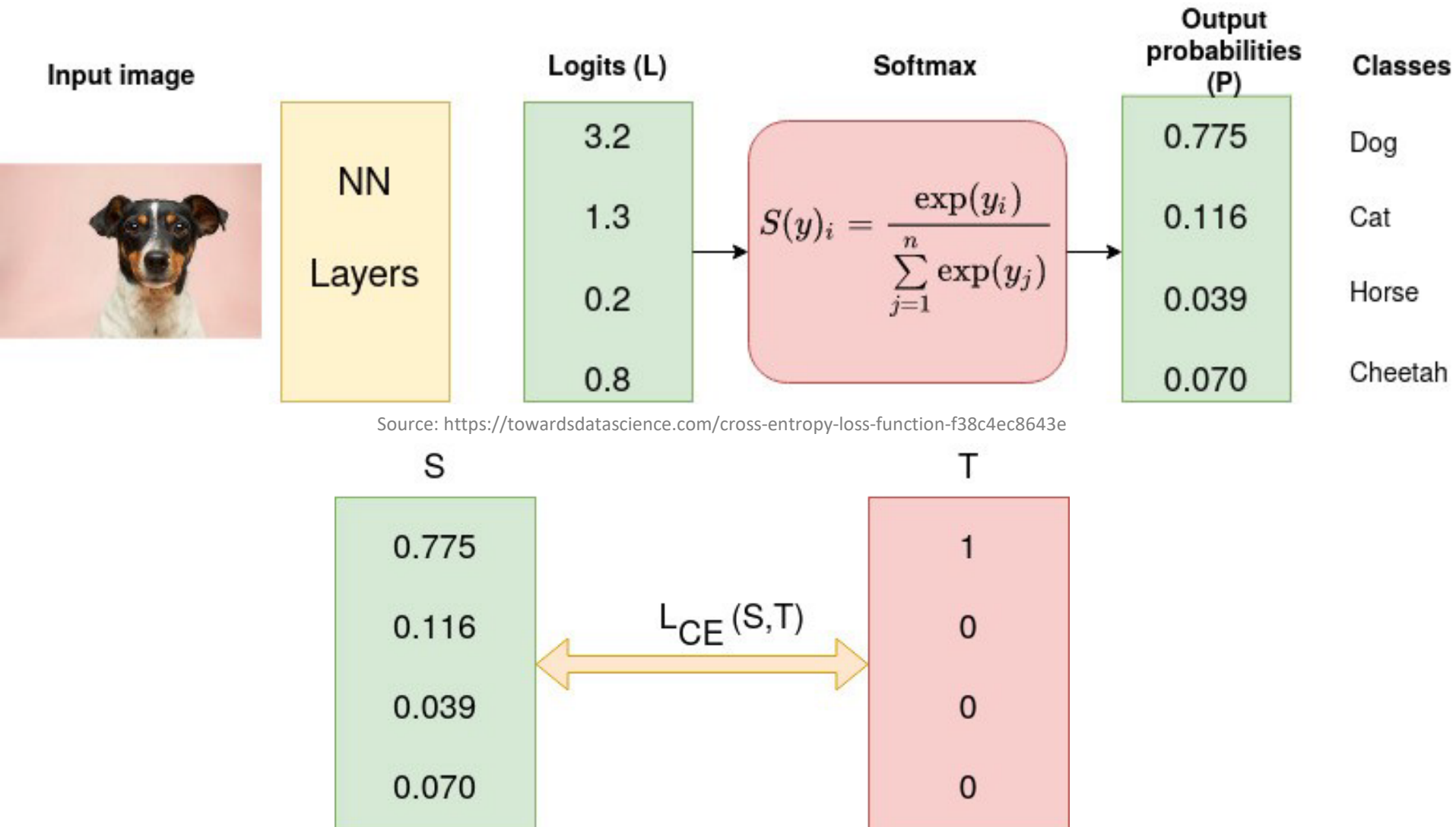
$$\text{Loss} = - \sum_{i=1}^{\text{output size}} y_i \cdot \log \hat{y}_i$$

Output size is the **number of classes** (categories)

in this classification task.

Apply to multi-class problem

Example:



Source: <https://towardsdatascience.com/cross-entropy-loss-function-f38c4ec8643e>

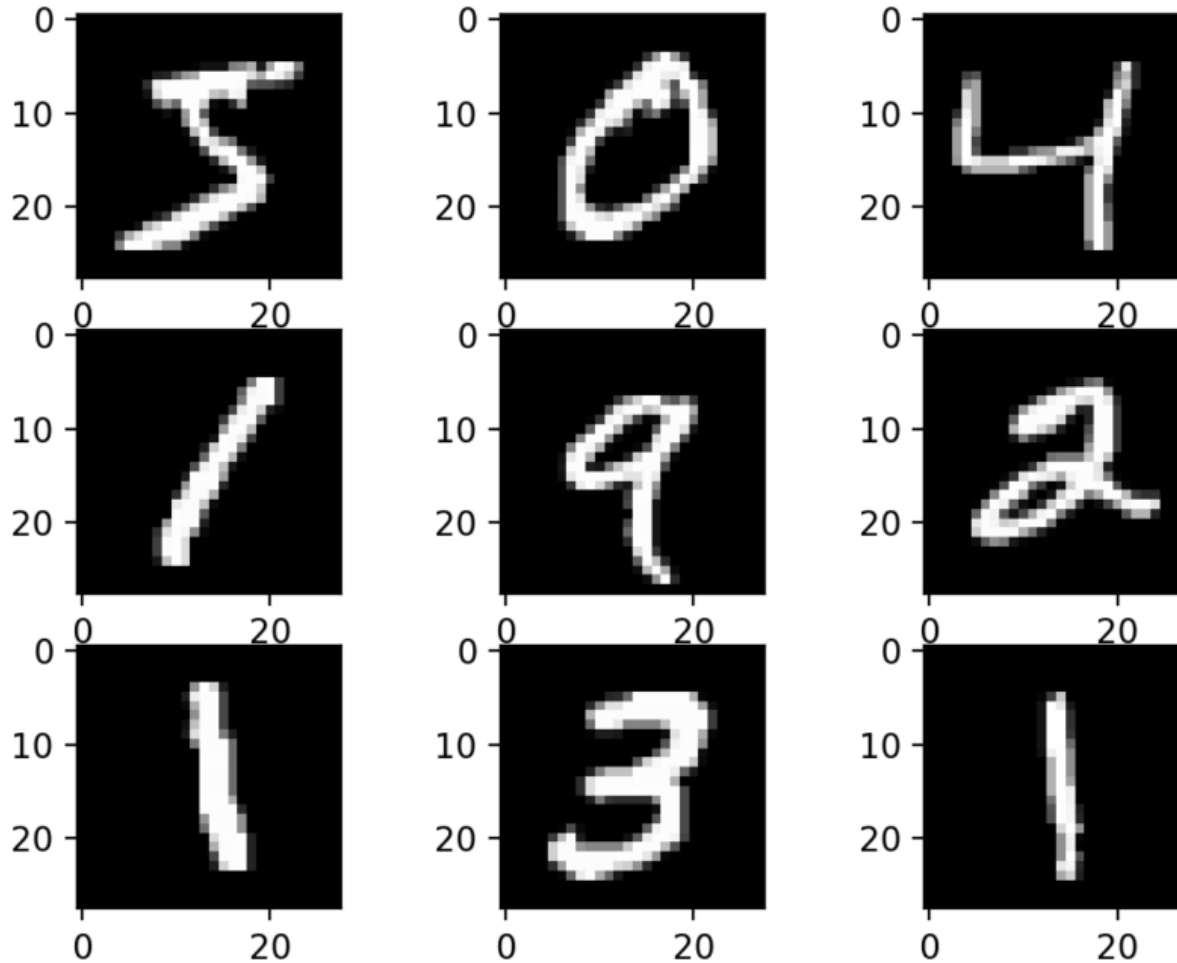
TensorFlow Playground : <https://playground.tensorflow.org/>



Example: MNIST



Data: 28*28 black-white images with label

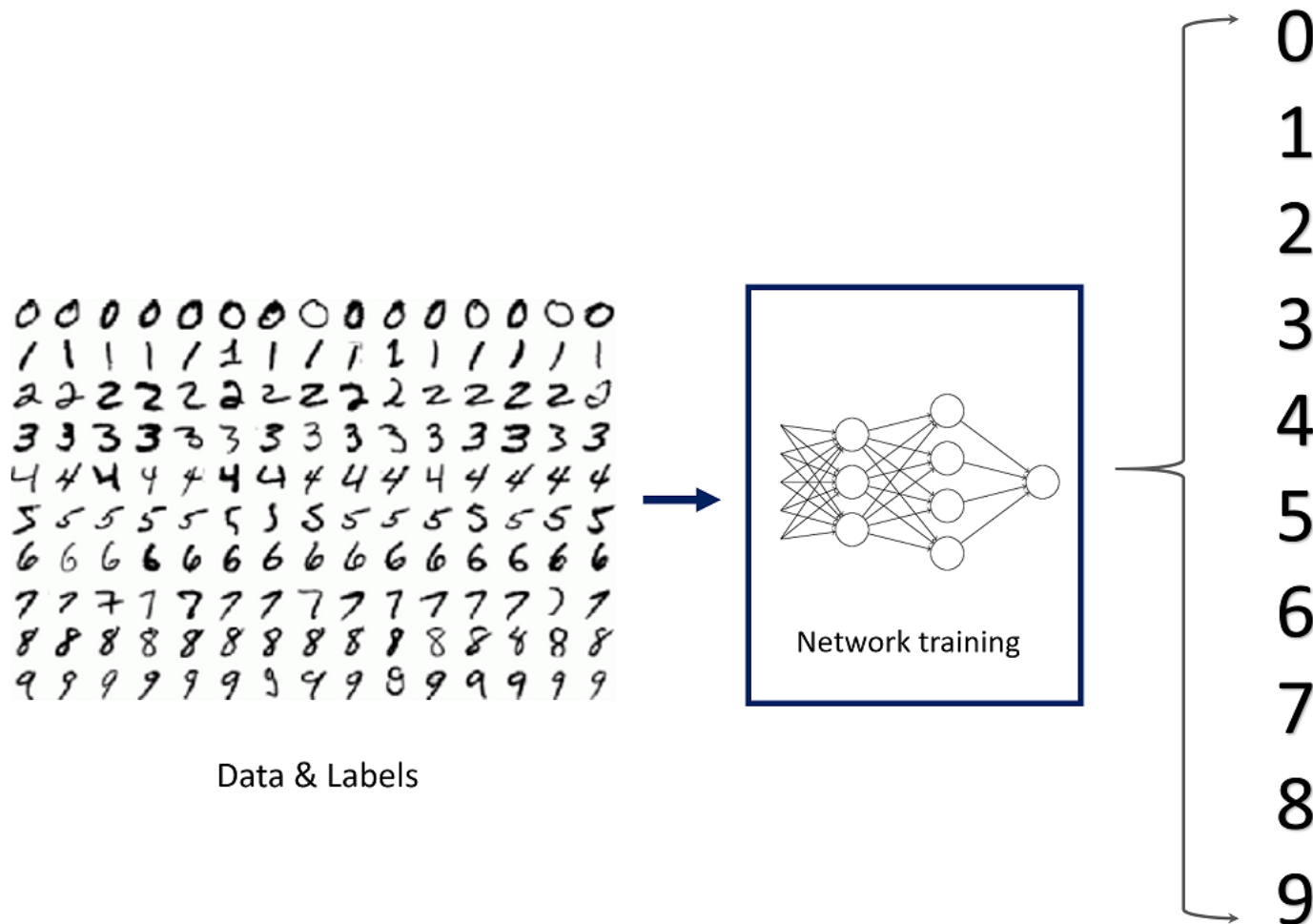


Source: <https://machinelearningmastery.com/how-to-develop-a-convolutional-neural-network-from-scratch-for-mnist-handwritten-digit-classification/#:~:text=MNIST%20Handwritten%20Digit%20Classification%20Dataset,-The%20MNIST%20dataset&text=It%20is%20a%20dataset%20of,from%200%20to%209%2C%20inclusively.>

Example: MNIST



Task: Train Classifier based on 60,000 training data
Test the trained classifier on 10,000 testing data



A closer look at the data

```
i, data = next(enumerate(trainloader))  
images, labels = data  
  
print(images)  
print(labels)
```

A closer look at the data

```
tensor([[[[-1., -1., -1., ..., -1., -1., -1.],  
          [-1., -1., -1., ..., -1., -1., -1.],  
          [-1., -1., -1., ..., -1., -1., -1.],  
          ...,  
          [-1., -1., -1., ..., -1., -1., -1.],  
          [-1., -1., -1., ..., -1., -1., -1.],  
          [-1., -1., -1., ..., -1., -1., -1.]]]],
```

```
[[[-1., -1., -1., ..., -1., -1., -1.],  
  [-1., -1., -1., ..., -1., -1., -1.],  
  [-1., -1., -1., ..., -1., -1., -1.],  
  ...,  
  [-1., -1., -1., ..., -1., -1., -1.],  
  [-1., -1., -1., ..., -1., -1., -1.],  
  [-1., -1., -1., ..., -1., -1., -1.]]]],
```

```
[[[-1., -1., -1., ..., -1., -1., -1.],  
  [-1., -1., -1., ..., -1., -1., -1.],  
  [-1., -1., -1., ..., -1., -1., -1.],  
  ...,  
  [-1., -1., -1., ..., -1., -1., -1.],  
  [-1., -1., -1., ..., -1., -1., -1.],  
  [-1., -1., -1., ..., -1., -1., -1.]]]],
```

```
tensor([0, 2, 9, 6, 7, 8, 8, 1, 7, 4, 7, 1, 7, 0, 3, 1, 8, 2, 9, 7, 9, 0, 8, 3,  
        1, 7, 7, 4, 0, 1, 0, 5])
```

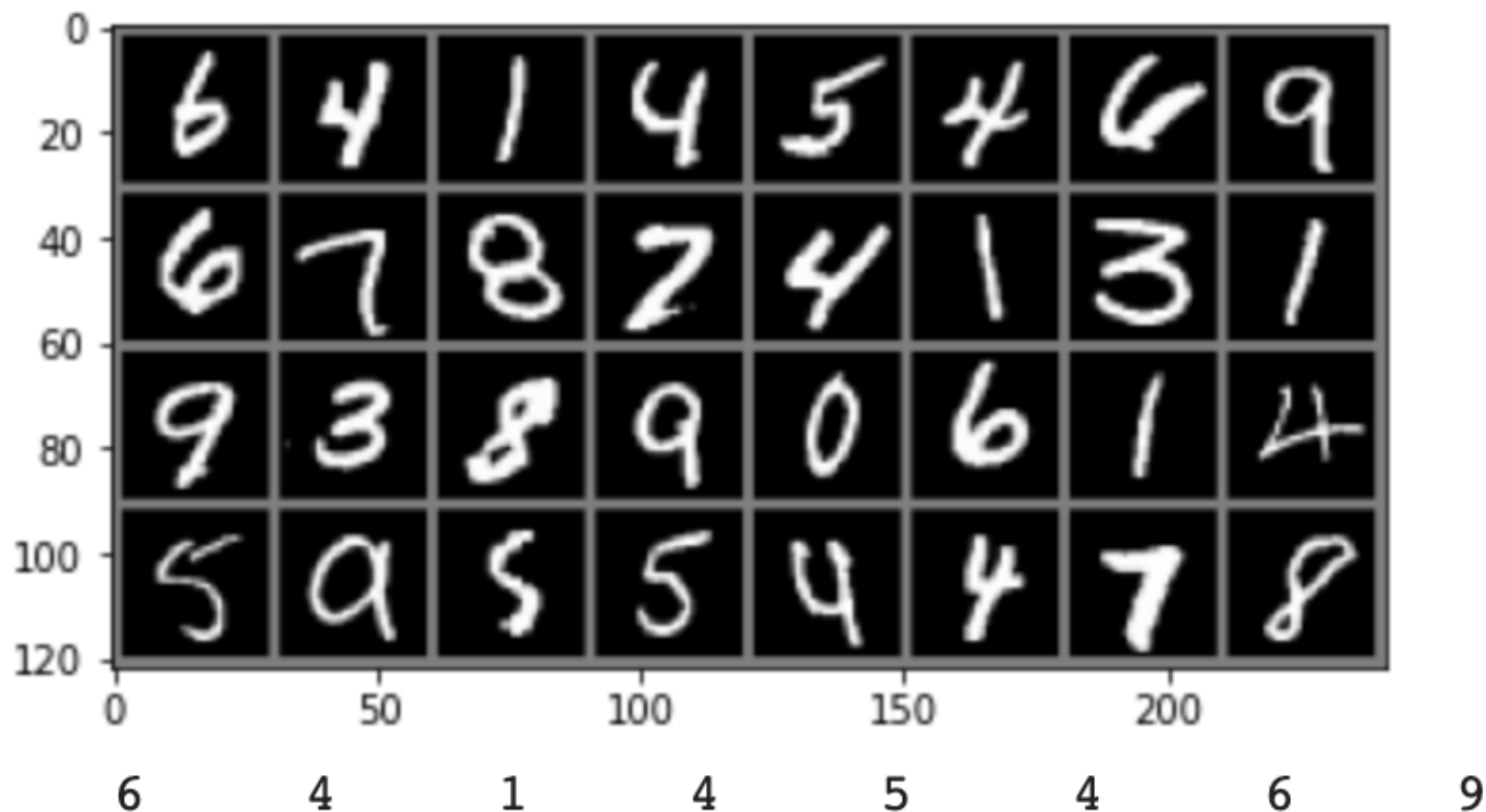
A closer look at the data

```
def imshow(img):
    img = img / 2 + 0.5 # Unnormalize the image
    npimg = img.numpy()
    plt.imshow(np.transpose(npimg, (1, 2, 0)))
    plt.show()

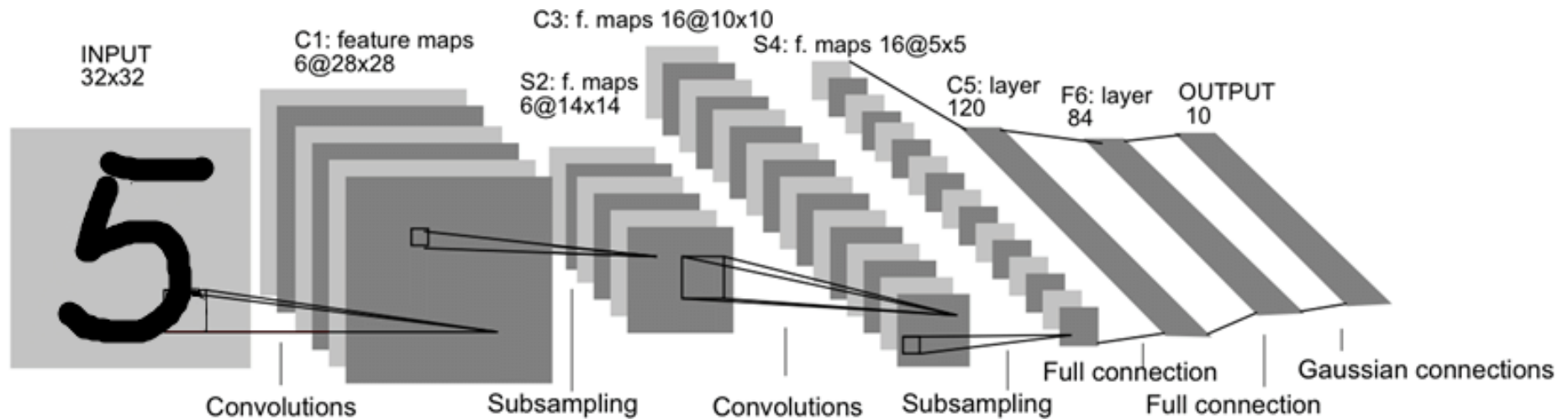
# Get some random training images
dataiter = iter(trainloader)
images, labels = next(dataiter)

# Show the images
imshow(torchvision.utils.make_grid(images))
# Print the labels
print(' '.join('%5s' % classes[labels[j]] for j in range(8)))
```

A closer look at the data



Build the Le-Net5



An early (Le-Net5) Convolutional Neural Network design, LeNet-5, used for recognition of digits

Source: https://www.researchgate.net/publication/321665783_Distributed_learning_of_CNNs_on_heterogeneous_CPUGPU_architectures

Build the Le-Net5

```
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(1, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 4 * 4, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 16 * 4 * 4)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x
```

Cross entropy loss function

```
import torch.optim as optim

# Use the cross entropy loss function
criterion = nn.CrossEntropyLoss()

# Use the cross SGD algorithm
optimizer = optim.SGD(net.parameters(), lr=0.001, momentum=0.9)
```

CROSSENTROPYLOSS

```
CLASS torch.nn.CrossEntropyLoss(weight=None, size_average=None, ignore_index=- 100,  
                                reduce=None, reduction='mean', label_smoothing=0.0) \[SOURCE\]
```

Train and test the LeNet-5 on MNIST dataset

Epoch 1: train loss = 1.088, test loss = 0.216, test accuracy = 93.53%
Epoch 2: train loss = 0.165, test loss = 0.102, test accuracy = 96.76%
Epoch 3: train loss = 0.102, test loss = 0.081, test accuracy = 97.41%
Epoch 4: train loss = 0.082, test loss = 0.070, test accuracy = 97.83%
Epoch 5: train loss = 0.068, test loss = 0.058, test accuracy = 98.15%
Epoch 6: train loss = 0.059, test loss = 0.057, test accuracy = 98.14%
Epoch 7: train loss = 0.053, test loss = 0.051, test accuracy = 98.26%
Epoch 8: train loss = 0.048, test loss = 0.047, test accuracy = 98.43%
Epoch 9: train loss = 0.043, test loss = 0.058, test accuracy = 98.10%
Epoch 10: train loss = 0.039, test loss = 0.043, test accuracy = 98.60%

